

Software Engineering (IT632)

Core Course (3-0-2-4)

MScIT Sem-II

Prerequisites

Students should have familiarity with data structures and basic algorithms, and exposure to at least one mainstream programming language.

Course Description

This course introduces the fundamental principles, concepts, and practices of Software Engineering as an engineering discipline. It emphasizes systematic approaches to software development, maintenance, and evolution, preparing postgraduate students to design, develop, and manage complex software systems in professional and research environments.

Course Objectives

- To develop a principled understanding of software engineering as a systematic, disciplined, and quantifiable engineering practice, enabling students to distinguish between ad hoc programming and professionally engineered software systems.
- To equip students with the ability to analyze, model, and specify software requirements rigorously, accounting for technical constraints, stakeholder needs, and socio-technical factors inherent in real-world systems.
- To cultivate competence in applying architectural and design principles to construct high-quality software systems, with explicit consideration of trade-offs among performance, scalability, maintainability, security, and cost.
- To instill an engineering mindset for ensuring software quality through testing, measurement, reviews, and controlled change, emphasizing risk management and long-term system evolution rather than short-term functionality.
- To foster ethical, professional, and managerial responsibility in software engineering practice, preparing students to make informed decisions that recognize the societal, legal, and economic impacts of software systems.

Key Topics

- Introduction to Software Engineering
- Software Processes and Lifecycle Models
- Iterative and Agile Development
- Requirements Engineering Fundamentals
- Requirements Elicitation and Specification
- System Modeling and Software Architecture

- Software Design Principles
- Software Construction Practices
- Software Testing Fundamentals
- Software Quality and Metrics
- Configuration and Change Management
- Software Project Management
- Emerging Trends in SE

Grading Components

- Class test, In-sem & End-sem exam, lab assignments, group project.

Evaluation Scheme

- **Lab(50%)**
 - Assignments & Labs: 20%
 - Team Project: 30%
- **Theory(50%)**
 - Class Tests: 10%
 - In-Sem Exam: 20%
 - End-Sem Exam: 20%

Course Outcomes

- Analyze complex software problems and justify the need for systematic software engineering approaches by distinguishing between programming-centric solutions and engineering-driven processes applicable to large-scale, long-lived systems.
- Elicit, analyze, and formally specify software requirements by applying appropriate requirements engineering techniques while managing ambiguity, stakeholder conflicts, and evolving constraints.
- Design software architectures and detailed designs that balance competing quality attributes, demonstrating informed trade-off decisions related to performance, scalability, maintainability, security, and cost.
- Apply software quality assurance techniques—including testing, reviews, metrics, and configuration management—to evaluate and improve software artifacts, emphasizing defect prevention and controlled system evolution.
- Estimate, plan, and manage software projects using foundational project management principles, including effort estimation, scheduling, risk assessment, and team coordination.
- Critically evaluate software engineering practices, processes, and tools in real-world and emerging contexts, including agile development, DevOps, and AI-assisted software engineering, recognizing their limitations as well as benefits.

P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
X	X	X	X	X			X	X			